

Phase 1 實作 Spec : Day 1-5 (繁體中文)

對話式 ERP 基建週 跑通：自然語言「幫我訂 1000 個 M6 給中鋼」→ AI 反問補欄位 → Confirm Card
→ DB 建單 → 可 undo

前置文件：[CONVERSATIONAL_ERP_DESIGN_ZH.md](#) (必讀) 版本：v1.0 (2026-05-15)



目錄

- [1. Phase 1 總目標 + 成功標準](#)
- [2. Day 1 : Tool Registry + Risk-Tier 框架](#)
- [3. Day 2 : ConfirmCard Schema + 前端元件](#)
- [4. Day 3 : 第一個 hard-write tool](#)
- [5. Day 4 : Slot-filling 反問機制](#)
- [6. Day 5 : E2E Demo + 錄影](#)
- [7. 跨日通用 Test 計畫](#)
- [8. Definition of Done](#)

1. Phase 1 總目標

5 天結束時必須能 demo 給老闆看的劇本：

```
[打開 Ouvoca 桌機 UI，登入 admin]
[點左側「AI 助手」]

User type: 「幫我訂 1000 個 M6 給中鋼」

[AI 在 2 秒內回 inline Card：
  📄 確認建立採購單

  供應商：中鋼公司 (S-001)
  零件：M6-BOLT-20 不銹鋼螺絲 20mm
  數量：1000
  單價：NT$ 0.5 (上次成交價)
  總額：NT$ 500

  [✓ 確認] [↩ 修改] [X 取消]
]

[使用者按「確認」]

[AI 回：✅ 已建立 PO-2026-105。預計 7 天到貨。
  若要取消，5 分鐘內說「取消剛才那筆」。
  點此查看 → /purchase/orders/PO-2026-105
]

[使用者再說：「取消剛才那筆」]

[AI 回：✅ 已撤銷 PO-2026-105。資料庫狀態回復到建單前。]

[使用者開瀏覽器查 /purchase/orders，確實看不到 PO-105]
```

5 天成功 = 上面 7 個 [...] 步驟全部跑通。沒跑通任一步就不算過 Phase 1。

Day 1 : Tool Registry + Risk-Tier

目標

建立統一的 tool 註冊機制，讓每個 tool 都帶 `risk_tier` / `slots` / `required_permission` metadata。改造現有 26 個 tool 接入此 registry。

交付清單

File	動作	用途
<code>backend/app/agents/registry.py</code>	NEW	<code>@register_tool</code> decorator + ToolMeta dataclass + 查詢 API
<code>backend/app/agents/__init__.py</code>	MODIFY	export registry
<code>backend/app/agents/domains/*_tools.py</code>	MODIFY × 10	每個 tool 加 <code>@register_tool</code> decorator
<code>backend/app/agents/engine.py</code>	MODIFY	從 registry 取 tool 而非 hardcoded list
<code>backend/tests/smoke/test_tool_registry.py</code>	NEW	驗證所有 tool 都註冊 + risk_tier 合法

registry.py 介面契約

```

from dataclasses import dataclass, field
from typing import Literal, Callable, Awaitable, Any
from enum import Enum

class RiskTier(Enum):
    READ = "read"
    SOFT_WRITE = "soft-write"
    HARD_WRITE = "hard-write"

@dataclass
class ToolMeta:
    name: str # 例: create_purchase_order
    domain: str # purchase
    risk_tier: RiskTier
    description: str # LLM 看的 (自然語言)
    slots: list[dict] = field(default_factory=list)
    # slot = { name: "qty", type: "int", required: True, description: "..."}
    required_permission: str | None = None # RBAC 權限 code
    func: Callable[..., Awaitable[Any]] = None # 實際執行函式
    undo_recipe: Callable | None = None # 若 soft/hard-write, 提供 undo

# 全域 registry
_REGISTRY: dict[str, ToolMeta] = {}

def register_tool(
    name: str,
    domain: str,
    risk_tier: RiskTier,
    description: str,
    slots: list[dict] = None,
    required_permission: str | None = None,
    undo_recipe: Callable | None = None,
):
    def decorator(func):
        _REGISTRY[name] = ToolMeta(
            name=name, domain=domain, risk_tier=risk_tier,
            description=description, slots=slots or [],
            required_permission=required_permission,
            func=func, undo_recipe=undo_recipe,
        )
        return func
    return decorator

def get_tool(name: str) -> ToolMeta | None:
    return _REGISTRY.get(name)

def list_tools(domain: str | None = None, tier: RiskTier | None = None) -> list[ToolMeta]:
    """給 AI engine 用, 列出 (過濾) 可用 tool。"""
    items = list(_REGISTRY.values())
    if domain:
        items = [t for t in items if t.domain == domain]
    if tier:

```

```

    items = [t for t in items if t.risk_tier == tier]
    return items

```

改造範例：現有 inventory_tools.py

```

# Before
async def _query_inventory(db, user, part_no: str = None, ...): ...

# After
from app.agents.registry import register_tool, RiskTier

@register_tool(
    name="query_inventory",
    domain="inventory",
    risk_tier=RiskTier.READ,
    description="查詢零件的庫存數量。輸入 part_no 或 part_id 之一。",
    slots=[
        {"name": "part_no", "type": "str", "required": False,
         "description": "料號，如 M6-BOLT-20"},
        {"name": "part_id", "type": "str", "required": False,
         "description": "UUID 形式的 part id"},
    ],
    required_permission="inventory.part.read",
)
async def _query_inventory(db, user, part_no: str = None, part_id: str = None):
    ...

```

Day 1 驗收 (Acceptance Criteria)

- ☐ `app/agents/registry.py` 存在 + 26 個 tool 全用 decorator 註冊
- ☐ `engine.py` 取 tool 走 `get_tool()` / `list_tools()` 不再 hardcode
- ☐ `python -c "from app.agents.registry import list_tools; print(len(list_tools()))"` 印 26
- ☐ `pytest tests/smoke/test_tool_registry.py` 全綠 (10 個案例：每個 domain 至少 1 個 tool / 所有 hard-write 必有 `required_permission` / ...)
- ☐ `bash scripts/run_gates.sh` 8/8 綠

Day 2 : ConfirmCard Schema + 前端元件

目標

定義 hard-write tool 回應的 JSON schema，並在 frontend Chat 頁實作 inline `<ConfirmCard>` 元件。

交付清單

File	動作	用途
backend/app/agents/confirm_card.py	NEW	Pydantic schema for ConfirmCardResponse
backend/app/api/chat.py	MODIFY	路由 hard-write 回 ConfirmCard 而非執行
frontend- desktop/src/components/ConfirmCard.tsx	NEW	React 元件
frontend- desktop/src/components/ChatMessage.tsx	MODIFY	偵測 type=confirm_required → 渲染 Card
frontend-desktop/src/pages/Chat.tsx	MODIFY	confirm button → 二次呼叫 API 帶 confirm_token
backend/tests/smoke/test_confirm_card.py	NEW	schema 驗證 + 流程測試

Schema 定義

```
# backend/app/agents/confirm_card.py
from pydantic import BaseModel
from typing import Literal, Any
from datetime import datetime

class ConfirmCardResponse(BaseModel):
    type: Literal["confirm_required"] = "confirm_required"
    confirm_token: str          # 後端產 uuid，前端要傳回來
    summary_zh: str             # 中文摘要
    summary_en: str             # 英文摘要
    action: str                 # 要執行的 tool name
    args: dict[str, Any]        # tool 的參數
    risk_tier: str              # "hard-write"
    undo_eligible: bool = True
    buttons: list[str] = ["confirm", "adjust", "cancel"]
    expires_at: datetime        # 5 分鐘後過期
    explanation: str | None = None # AI 的補充說明
```

前端元件介面

```
// frontend-desktop/src/components/ConfirmCard.tsx
interface ConfirmCardProps {
  confirm_token: string
  summary: string          // i18n 後的版本
  action: string
  args: Record<string, any>
  expires_at: Date
  on_confirm: () => void
  on_adjust: () => void
  on_cancel: () => void
}

export function ConfirmCard(props: ConfirmCardProps) {
  // 視覺：黃色框 + 圖示 + 摘要 + 三顆按鈕 + 倒數計時
}
```

Day 2 驗收

- ☐ `pytest tests/smoke/test_confirm_card.py` 全綠
- ☐ 在前端 Chat 頁，硬編一個 dummy hard-write tool 回應，能渲染出 Card (含 3 顆按鈕 + 倒數)
- ☐ 按「取消」→ Card 消失、聊天記錄留下「已取消」訊息
- ☐ 按「確認」→ 跳出「執行中...」狀態 (Day 3 才接真執行)
- ☐ `bash scripts/run_gates.sh` 8/8 綠

Day 3 : 第一個 Hard-Write Tool — `create_purchase_order_with_confirm`

目標

真正接通：自然語言「幫我下個 PO」→ ConfirmCard → 確認 → DB 真的有 PO。

交付清單

File	動作	用途
<code>backend/app/agents/domains/purchase_write_tools.py</code>	NEW	<code>create_purchase_order</code> 寫入 tool
<code>backend/app/agents/engine.py</code>	MODIFY	處理 <code>confirm_token</code> : 第二次 call 時實際執行
<code>backend/tests/integration/test_create_po_via_chat.py</code>	NEW	端到端整合測試

Tool 實作骨架

```
@register_tool(
    name="create_purchase_order",
    domain="purchase",
    risk_tier=RiskTier.HARD_WRITE,
    description="建立採購單。需提供供應商和品項清單。",
    slots=[
        {"name": "supplier", "type": "str", "required": True,
         "description": "供應商名稱或代碼"},
        {"name": "items", "type": "list", "required": True,
         "description": "品項: [{part_no, qty, unit_price?}]"},
    ],
    required_permission="purchase.order.create",
    undo_recipe="cancel_purchase_order",
)

async def create_purchase_order(
    db, user, *,
    supplier: str, items: list[dict],
    confirmed: bool = False, confirm_token: str | None = None,
):
    # 1. 解析 supplier (用 entity-resolver)
    s_candidates = await search_supplier(db, supplier)
    if len(s_candidates) > 1:
        return DisambiguationResponse(...)
    if not s_candidates:
        return {"error": f"找不到供應商「{supplier}」"}
    supplier_obj = s_candidates[0]

    # 2. 解析 items
    resolved_items = []
    for it in items:
        part = await get_part_by_no(db, it["part_no"])
        if not part:
            return {"error": f"找不到料號「{it['part_no']}」"}
        # 缺 unit_price → fetch last
        if "unit_price" not in it:
            it["unit_price"] = await last_supplier_price(db, supplier_obj.id, part.id) or
part.unit_cost
        resolved_items.append(**it, "part_id": part.id)

    total = sum(it["qty"] * it["unit_price"] for it in resolved_items)

    # 3. 還沒確認 → 回 ConfirmCard
    if not confirmed:
        token = await issue_confirm_token(
            user_id=user.user_id,
            action="create_purchase_order",
            args={"supplier_id": supplier_obj.id, "items": resolved_items},
        )
        return ConfirmCardResponse(
            confirm_token=token,
            summary_zh=f"建立 PO 給 {supplier_obj.name} · "
                f"{len(resolved_items)} 項 · NT$ {total:.0f}",
```



```

summary_en=f"Create PO to {supplier_obj.name} · "
            f"{len(resolved_items)} items · NT$ {total:.0f}",
action="create_purchase_order",
args={"supplier_id": supplier_obj.id, "items": resolved_items},
risk_tier="hard-write",
expires_at=now() + timedelta(minutes=5),
).dict()

# 4. 已確認 → 真的執行
await validate_confirm_token(token=confirm_token, user_id=user.user_id)
po = await purchase_service.create_purchase_order(
    db, {"supplier_id": supplier_obj.id, "items": resolved_items},
    user=user.raw_user,
)
# 5. 寫 ActionHistory (給 undo 用)
await action_history.write(
    session_id=user.session_id, user_id=user.user_id,
    tool="create_purchase_order",
    args_after={"po_id": po.id, "po_no": po.po_no},
    undo_recipe=f"cancel_purchase_order(po_id='{po.id}')",
    expires_minutes=5,
)
return {
    "ok": True,
    "po_no": po.po_no,
    "po_id": po.id,
    "message": f"已建立 PO {po.po_no}",
}

```

Day 3 驗收

- ☐ 整合測試：用 admin token 對 `/api/chat-v2` 連發 2 次 message
 - 第 1 次：「給中鋼下 PO 1000 個 M6-BOLT-20」→ 回 ConfirmCard JSON
 - 第 2 次：帶 `confirm_token` 確認 → DB 多一筆 PO
- ☐ 整合測試：第 2 次過期 token → 回 410 Gone
- ☐ 沒權限的 user → 回「您沒有 purchase.order.create 權限」
- ☐ `bash scripts/run_gates.sh` 8/8 綠

Day 4 : Slot-filling 反問

目標

缺欄位時 AI 主動問。讓使用者不必一次講完整句。

交付清單

File	動作	用途
<code>backend/app/agents/slot_filling.py</code>	NEW	找缺欄位 + 產生反問 prompt
<code>backend/app/agents/engine.py</code>	MODIFY	在 tool call 前先用 slot-filling 檢查
<code>backend/app/models/ai_governance.py</code>	MODIFY	DecisionLog 加 <code>slots_state</code> JSON 欄位
<code>backend/tests/integration/test_slot_filling.py</code>	NEW	5 個情境的反問測試

反問策略

LLM prompt template :

你是 Ouvoca 採購助手。使用者要建 PO，但缺少必要資訊：

- 缺：`{missing_slots}`
- 已知：`{filled_slots}`

請用一句中文 (< 30 字) 友善地問出缺少的資訊。

若多個缺欄位，一次只問最重要的一個。

不要解釋、不要說「請告訴我」這類官腔。

Day 4 驗收

- ☐ 整合測試 5 個情境：
 - 「幫我建 PO」→ AI 問「給哪家供應商？」
 - 「給中鋼建 PO」→ AI 問「要訂什麼料號？」
 - 「給中鋼訂 M6」→ AI 問「數量多少？」
 - 「給中鋼訂 M6 1000 個」→ AI 進入 disambiguation (3 個 M6 候選)
 - 「給中鋼訂 M6-BOLT-20 1000 個」→ AI 直接回 ConfirmCard
- ☐ `bash scripts/run_gates.sh` 8/8 綠

Day 5 : E2E Demo + 錄影

目標

串完整個劇本，錄一段 30-60 秒影片，存進 `docs/demos/`，作為對外行銷素材。

工作項目

工作	詳細
整合測試最終版	1 個大 test，跑完 §1 的 7 個劇本步驟
演練流程腳本	<code>scripts/demo/conversational_po.md</code> 用者照著按
螢幕錄影	OBS 或 Windows Game Bar 錄 30-60 秒
影片存檔	<code>docs/demos/phase1_conversational_po.mp4</code> (or .gif)
WORKLOG 更新	寫上完整 Phase 1 成果
README 加 demo link	「Conversational ERP demo」放最顯眼處

Demo 腳本 (給錄影時照念 / 照做)

```
0:00 [ 桌機開瀏覽器、輸入 http://localhost:5173 ]
0:03 [ 登入 admin / admin123 ]
0:05 [ 點左側 AI 助手 ]
0:08 旁白：「Ouvoca 不需教育訓練 — 老闆只要講話。」
0:12 [ 鍵入：給中鋼下 PO ]
0:15 [ AI 回：「要訂什麼料號？」 ]
0:17 [ 鍵入：M6 1000 個 ]
0:20 [ AI 回 Disambiguation：M6-BOLT-20？M6-NUT？M6-WASHER？ ]
0:25 [ 點 M6-BOLT-20 ]
0:28 [ AI 回 ConfirmCard：金額 NT$500、3 顆按鈕 ]
0:32 旁白：「中等風險動作必須確認 — 絕不誤觸。」
0:36 [ 點「確認」 ]
0:40 [ AI 回：✅ 已建立 PO-2026-105 ]
0:42 旁白：「5 分鐘內可以後悔。」
0:45 [ 鍵入：取消剛才那筆 ]
0:48 [ AI 回：✅ 已撤銷 ]
0:52 [ 切到瀏覽器 /purchase/orders，PO 真的不見 ]
0:58 旁白：「自然語言操作 ERP。教育訓練 2 小時上手。Ouvoca。」
1:00 [ Logo + Tagline ]
```

Day 5 驗收

- ☐ 影片產出 (30-60 秒，包含 §1 的 7 步劇本)
- ☐ `tests/integration/test_e2e_conversational_po.py` 一個大 test，跑完 7 步全綠
- ☐ WORKLOG #17 詳細記錄 Phase 1 成果
- ☐ CLAUDE.md 升 v2.8 → v2.9
- ☐ README 加 demo 區塊 (gif 或 video 嵌入)
- ☐ commit + push + CI 綠燈

7. 跨日通用 Test 計畫

每個 Day 都要：

```
# 跑既有 138 tests + Day N 新加的 tests
cd backend && python -m pytest tests/ -q --tb=line

# 跑完整 gate
bash scripts/run_gates.sh      # 必須 8/8 綠
```

每天都不能讓既有測試退化。

新增測試矩陣（5 天累計）：

Day	新增 tests	累計
1	tool_registry × 10	148
2	confirm_card × 8	156
3	create_po_via_chat × 12	168
4	slot_filling × 5	173
5	e2e_conversational_po × 1 (大 test)	174

8. Definition of Done

Phase 1 結束時必須全部勾：

- ☐ \$1 的 7 步劇本 e2e 跑通 (pytest)
- ☐ 一段 30-60 秒 demo 影片存進 git
- ☐ `bash scripts/run_gates.sh` 全綠
- ☐ CI 綠燈
- ☐ WORKLOG #17 + CLAUDE.md v2.9 已 push
- ☐ 138 → 174 tests · 0 退化
- ☐ Phase 2 可以線性開展 (registry 框架已驗、之後加 tool 半天一個)

沒勾完 = Phase 1 沒完工。不可往 Phase 2 推進。



相關文件

- [架構設計 \(必讀前置 \)](#)
- 英文版 : `CONVERSATIONAL_ERP_PHASE1_SPEC_EN.md`

版本 : v1.0 · 最後更新 : 2026-05-15 · 狀態 : Phase 1 動工 spec